When Does Monitoring Long-Horizon Agents Lead to Better Performance?

Anonymous Author(s)
Affiliation
Address
email

Abstract

Long-horizon language-model agents can experience performance degradation as their interaction histories grow and context accumulates in their agent state [4]. We study how well different monitoring methods predict agent degradation, and whether compacting or reconstructing agent state improves downstream task performance. Across conversational reasoning tasks (Evolving-Intent GSM8K [3]) and multi-turn tool-use tasks (BFCL Multi-Turn [5]), we compare "active" observational monitors, "passive" monitors, and baseline context- and time-based approaches. We find that monitoring is not a free observation: active probes can alter the trajectories they are intended to measure, producing an "observer effect" where task success tends to decrease. We further find that the value of an intervention depends on how the agent state is recovered: lossy LLM-generated compaction can substantially reduce performance, whereas deterministic reconstruction can avoid this penalty and, for some monitor pairings, improve success relative to no intervention. These results suggest a taxonomy in which observation method, signal quality, and recovery mechanism are evaluated jointly, rather than treating monitoring as a standalone prediction problem.

1 Introduction

Modern agentic workflows have seen drastically increasing adoption for long-horizon tasks that require maintaining goals, constraints, and context over an extended period of time (i.e deep research, coding, etc) [2, 13]. As these trajectories grow longer, however, agents can gradually lose important context and start hallucinating incorrect information [3, 4]. A natural response is to monitor an agent during execution and intervene whenever its behavior starts to degrade. This can include restarting the agent compacting its context or simply restructuring state before the agent continues execution [7, 10, 14].

The need to preserve agent behavior creates an appealing intuition: if we can detect degradation earlier and more accurately, we should be able to intervene at better times and improve task performance. This has drastic implications for all agent tasks, whether it be coding benchmarks like SWE-bench [2] or enterprise workflows like τ-bench [12].

Much of agent monitoring can thus be viewed as a prediction problem: given an agent current behavior/state, determine whether it is likely to fail in the near future. Under this view, a more accurate monitor should naturally produce a better agent, assuming better timing positively influences intervention.

Let M_t denote a monitor alarm at turn t, $F_{>t}$ a subsequent failure, $\pi(M)$ the policy that acts on the alarm, and Y final task success. The usual prediction-first view assumes

$$(\Pr(M_t = 1 \mid F_{>t} = 1) \uparrow,\ \Pr(F_{>t} = 1 \mid M_t = 1) \uparrow) \quad \Longrightarrow \quad \mathbb{E}[Y \mid \pi(M)] \uparrow . \tag{1}$$

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

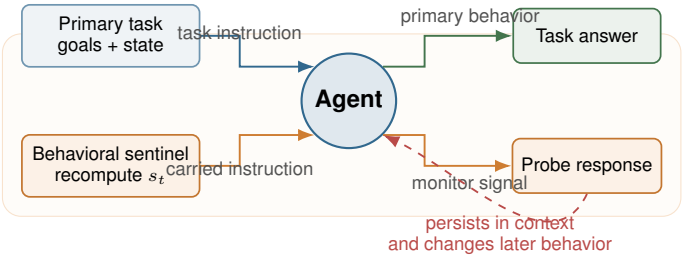

Figure 1: A behavioral sentinel is carried beside the primary task. Its response can warn of degradation, but the additional instruction and state also become part of the trajectory being measured.

We argue this view is incomplete; runtime monitors do not necessarily have to be a passive observer of an agent trajectory.

We explore a method of monitoring that introduces additional instructions or computations directly into the system prompt, separate from the original task we gave the agent.

We study one such class of monitors, which we call behavioral sentinels: canaries that an agent must maintain throughout a task and whose degradation can serve as a warning signal. For example, an agent might be asked to remember and update a small piece of state while simultaneously completing its primary task. The intuition here is if the agent fails to maintain the state before completing the underlying task, the sentinel can provide an early warning signal.

However, asking the agent to maintain such a signal can itself change the trajectory of an agent. Across our experiments, we find an "observer effect": a probe cannot predict the degradation of an agent without playing some part in the degradation itself. Monitoring therefore is not always a free source of information, and higher quality signals do not automatically imply higher performance.

Detecting degradations is useful because the resulting signal eventually triggers an intervention. Intuitively, this means the value of a monitor may depend not just on when it fires but what the system does afterwards. We study this interaction directly by evaluating monitoring under different recovery mechanisms. Specifically, to answer the question of whether the effect of monitoring depends on the recovery mechanism it is paired with, we compare recovery based on agent-generated context compaction with recovery that reconstructs state from an external source of truth.

Our experiments reveal that under lossy-agent generated compaction, carrying additional state can reduce the benefit of an otherwise well-timed intervention. However, when the state is reconstructed through deterministic regrounding, this penalty disappears and the monitoring setup can be beneficial.

Our work makes three primary contributions:

- We characterize monitoring as both active and passive, rather than a free observation. Active behavioral probes alter trajectories, producing an observer effect
- We experimentally isolate the interaction between monitoring and recovery approaches. Holding the monitoring setup fixed, we show that changing how agent state is reconstructed can change whether monitoring improves downstream performance.
- We evaluate monitoring as a complete systems problem. Across multiple models and long-horizon tasks, we find that no single monitoring strategy consistently dominates.

2 Related Works

2.1 What has already been done

Recent work has explored detecting degradation on agents before task completion

Doomed from the Start [6] introduces the idea of utilizing probes over internal model activations to predict task failure and terminate struggling trajectories. *TACT* [8] similarly detects agent drift from internal activation and uses activation steering to mitigate it during execution.

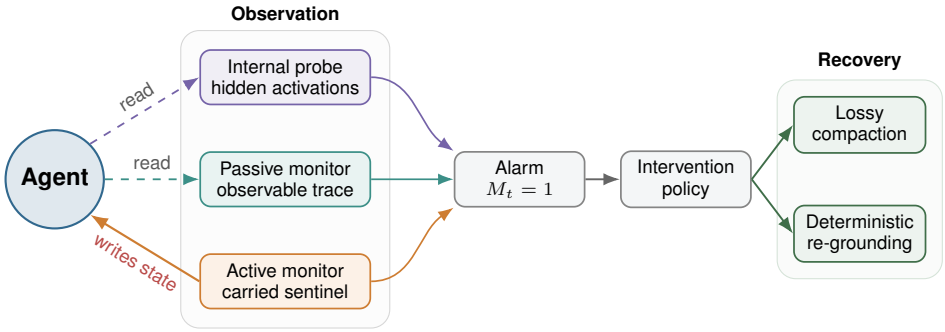

Figure 2: Three monitoring methods and their relationship to the agent. Internal probes read hidden activations, passive monitors read the observable trace, and active monitors write additional state into the trajectory. Each signal ultimately controls an intervention and recovery mechanism.

Other work has introduced explicit diagnostic probes: *Canary Tools* [1], for example, inserts adversarially designed tools into an agent's context to expose weaknesses in tool-selection reasoning.

Accurate Failure Prediction in Agents Does Not Imply Effective Failure Prevention [9] further shows that accurate failure prediction does not necessarily imply effective failure prevention when interventions disrupt otherwise successful trajectories.

Together, these methods show us that agent failure can often be detected before task completion, whether it be internal representations, observing trajectories, or introducing probes, but whether that leads to genuine performance improvement is still unclear.

2.2 What gap remains

A newer line of work explores what should happen after degradation is detected. *Fail-Fast, Restart-Smart* [10] combines early failure prediction with different restart strategies, showing us that preserving useful work across sessions can outperform a cold restart. *AgentRewind* [14] further treats recovery as a state management problem, checkpointing both agent context and environment state, so execution can return to an earlier point after agent failure.

Separately work on long-horizon context management has shown that LLM-generated compaction is inherently lossy and can unpredictably discard important information that is needed for later sessions [7].

2.3 Why this paper is different

Prior work therefore establishes two notions: agent failures can be monitored and that choosing the right recovery strategy matters. However, these components are typically evaluated separately. Monitoring work primarily asks whether a signal predicts a failure, while recovery work asks how best to restore or restart an agent once intervention occurs. To our knowledge, prior work has not studied how the monitoring mechanism itself interacts with agent state after intervention. We study this precise interaction explicitly. Using behavioral sentinels and passive monitors, we measure both signal quality and how observation changes task execution and recovery.

In particular, we hold the monitoring setup fixed while replacing lossy agent-generated compaction with deterministic external re-grounding, allowing us to precisely test whether the effect of monitoring changed depending on the recovery mechanism.

3 Experimental Setup

Our experiments follow a two-stage design. We first use a controlled study to isolate how monitoring, intervention timing, and recovery interact. We then evaluate whether these conclusions persist in a deployment-oriented study using larger models and realistic agent benchmarks.

Across both studies, we separate three components of an agent system:

Table 1: Controlled 2×2 mechanism study.

	No intervention	Oracle-timed intervention
No carried probe	A	B
Carried probe	C	D

Monitoring strategy: what signal is used to detect degradation?

Intervention policy: when does the system act on that signal?

Recovery mechanism: how is agent state reconstructed after intervention?

3.1 Controlled mechanism study

We first study the monitoring recovery interaction under controlled conditions using GPT-OSS-20B on long-horizon coding, register-tracking, and bAbI-style tasks [11]. These environments require intermediate progress to persist in agent state across multiple turns and allow us to identify when task performance begins to degrade.

Our primary active monitors are behavioral sentinels that require information that cannot simply be copied from the model's previous response. Accordingly, we construct probes whose correct response must be recomputed from the current state. For example, a probe can require a checksum over the current register values or a fresh arithmetic check derived from values in the current turn. Failure to maintain the sentinel provides a possible signal of underlying degradation.

Because these probes are carried directly inside the agent's context, observing the agent may itself change the trajectory being measured. We therefore compare otherwise identical trajectories with and without a carried probe.

Intervention timing. To separate monitor quality from the potential value of acting at the correct time, we compare trajectories with no intervention against trajectories receiving an oracle-timed intervention. Oracle timing uses knowledge of subsequent task degradation and is not intended as a deployable monitor. Instead, it provides a controlled estimate of the maximum performance that could be gained from favorable intervention timing.

For each task, we label the first hallucinated turn in the no-intervention trajectory and schedule the oracle reset immediately before that turn; tasks without a labeled hallucination do not receive an oracle reset.

We repeat the intervention conditions under two recovery mechanisms:

Agent-generated compaction asks the model to compress its accumulated context into a shorter representation before continuing execution. Because recovery depends on the model's existing trajectory, information that has already been forgotten, distorted, or deprioritized may not survive compaction.

External re-grounding instead reconstructs the relevant task state deterministically from an external source of truth. For example, a register-tracking task can replay the authoritative user-issued updates to rebuild every current register value. It does not depend on what information the model itself retained immediately before intervention.

Because no recovery occurs in conditions A or C, these conditions are shared across operators. The recovery mechanism affects only the intervention conditions B and D.

For recovery operator R, we measure whether carrying a probe changes the value of an otherwise well-timed intervention:

$$\Delta_{\text{interaction},R} = \big[P(D \mid R) - P(C)\big] - \big[P(B \mid R) - P(A)\big]. \tag{2}$$

3.2 Deployment-oriented evaluation

The controlled study is designed to isolate mechanisms, but it does not capture the full conditions of deployed agent monitoring. So we conduct a second set of experiments on BFCL Multi-Turn and

Evolving Intent GSM8K using DeepSeek V4 Flash, GPT-OSS-120B, GPT-5.6 Luna, and GPT-5.6 Terra.

Here, monitors operate online and must determine when intervention occurs without oracle timing.

We distinguish between monitors that directly interact with the agent during execution and monitors that observe the trajectory without modifying it.

Active monitors introduce additional instructions or state directly into the agent's context. Passive or zero-carry monitors obtain a signal without persisting additional monitoring state in the agent's trajectory. These include trace-based monitoring and forked evaluation strategies.

We evaluate these monitoring strategies under four post-intervention conditions:

1. no recovery,
2. LLM-generated context compaction,
3. deterministic public-state re-grounding, and
4. a bounded quote-only WATCH note.

In the controlled study, compaction asks the agent to summarize the state it believes is current. We then erase the conversation and continue from the task instructions and this model-written summary. In deployment, we use a deterministic lossy proxy: we erase the trace, restore the original instructions, and retain short excerpts from only the four latest messages (240 bytes each; 1,600 bytes total). Re-grounding also starts a fresh trace, but rebuilds state from external records rather than the agent's memory. For coding tasks, the harness replays the user's edits; in deployment, it restores the instructions plus an exact snapshot of completed public turns and metadata. WATCH keeps the full trace and appends an at-most-80-token note quoting observed evidence. No recovery leaves the monitored trajectory unchanged.

The recovery intervention study is restricted to GPT-5.6 Luna on Evolving Intent GSM8K; BFCL is used only for the monitoring evaluation.

Unlike the controlled mechanism study, monitoring methods act at their natural firing times in this setting. Intervention rates are not forced to match across methods, although each policy is limited to at most one intervention per task. We report intervention incidence alongside task success to distinguish differences in recovery quality from differences in how frequently a monitoring policy acts.

3.3 Evaluation metrics and protocol

We evaluate each monitoring system at two levels.

Monitoring quality. We report precision, recall, and area under the precision-recall curve (AUPRC) to measure how well a signal identifies subsequent task degradation.

End-to-end utility. We measure final task success, intervention frequency, token usage, latency, and monetary cost where available. These metrics capture whether a monitoring signal actually improves the complete agent system rather than just predicting failure accurately.

4 Results

4.1 Signal quality does not identify a universal monitor

Figure 3 compares precision, recall, and AUPRC at a matched firing rate. Active recomputation attains the highest AUPRC in three of seven model-by-task settings, while passive or baseline methods lead in the remaining four. The strongest observed signal is active recomputation on BFCL with GPT-OSS-120B (AUPRC 0.942), a gap of 0.058 from the perfect-label ceiling of 1.0; this ceiling is not a deployed oracle arm. Precision and recall expose a further tradeoff: the same signal has precision 1.00 but recall 0.20, so a clean alarm can still miss most failures. More generally, the identity of the best signal changes with both model and benchmark.

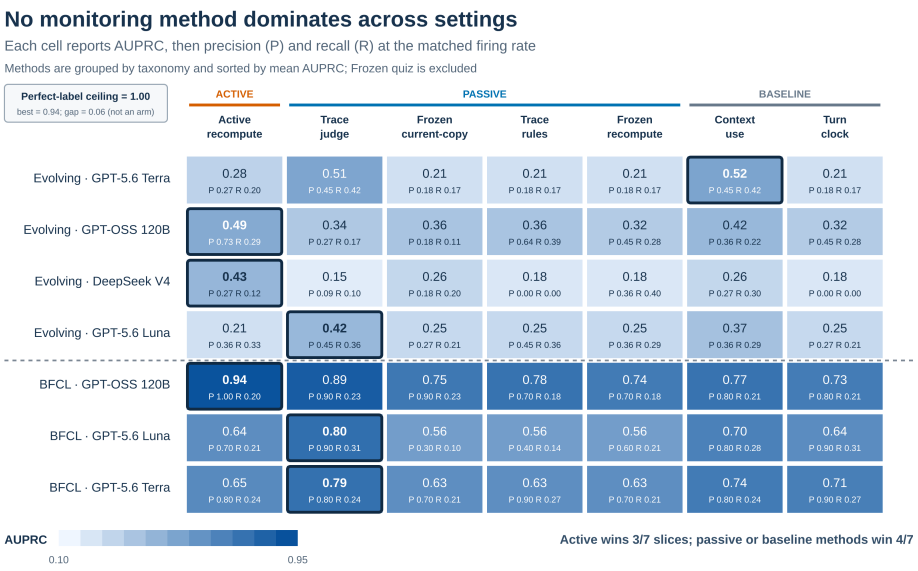

Figure 3: Signal quality across powered model-by-task settings. Each cell reports AUPRC and matched-rate precision and recall; outlined cells are within-setting AUPRC leaders. Evolving Intent uses verified final failure labels, while BFCL supplies turn-level action-failure evidence. Frozen quiz is excluded.

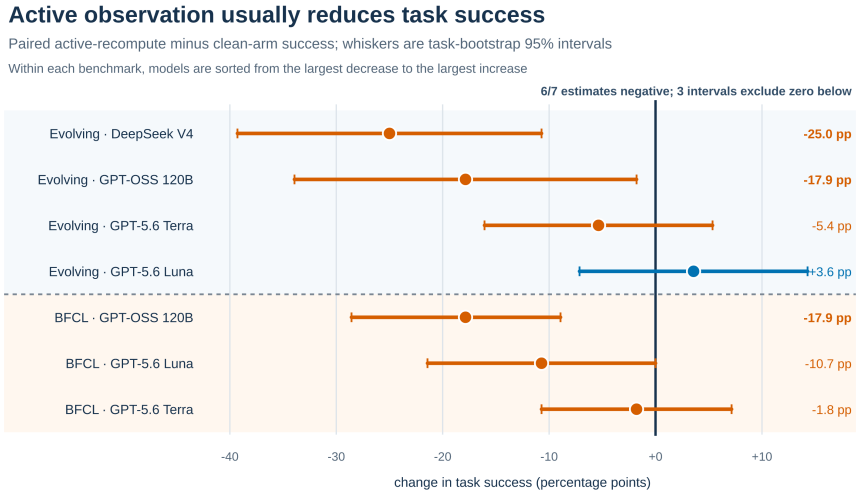

Figure 4: Observer effect for the prespecified active-recomputation contrast. Points are paired changes in task success relative to a clean arm on the same 56 tasks; whiskers are task-bootstrap 95% intervals.

4.2 Active monitoring changes the trajectory it measures

Signal quality alone does not price observation. Figure 4 compares otherwise matched clean and active-recomputation trajectories. Six of seven point estimates are negative, three paired 95% intervals exclude zero below, and the largest decrease is 25.0 percentage points for DeepSeek V4 on Evolving Intent. GPT-5.6 Luna on Evolving Intent is the only positive point estimate (+3.6 points), and its interval includes zero. The observer effect is therefore a strong trend, rather than a universal law.

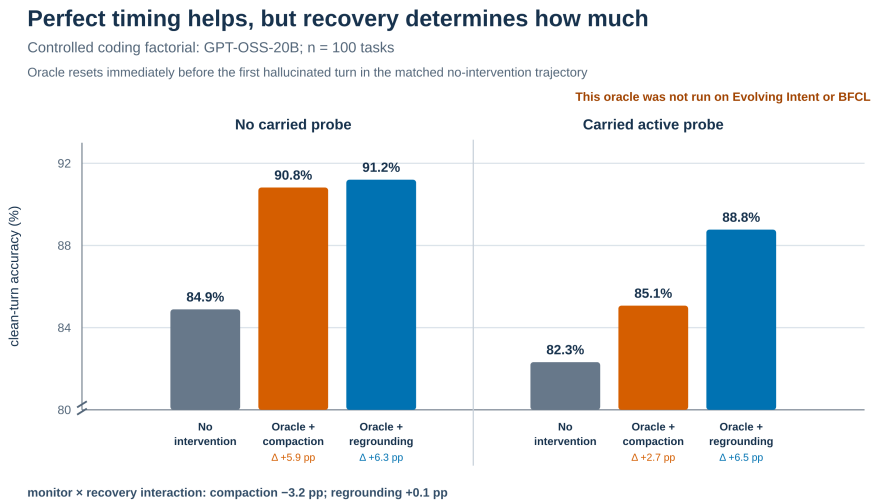

Figure 5: Oracle-timed intervention in the controlled coding factorial ($n = 100$ tasks). The oracle resets immediately before the first hallucinated turn from the matched no-intervention trajectory. The result measures clean-turn accuracy and should not be read as an Evolving Intent or BFCL oracle comparison.

4.3 Oracle timing reveals headroom, conditional on recovery

The controlled coding study isolates whether favorable timing can help at all. Without a carried probe, clean-turn accuracy increases from 0.849 with no intervention to 0.908 with oracle-timed compaction and 0.912 with oracle-timed re-grounding. With a carried probe, the corresponding gains are smaller under compaction ($0.823 \rightarrow 0.851$) but recover under re-grounding ($0.823 \rightarrow 0.888$). Consequently, the monitor-by-recovery interaction is -0.032 under compaction and $+0.001$ under re-grounding. This oracle was only defined for the GPT-OSS-20B controlled coding study: the Evolving Intent and BFCL deployment comparisons use natural monitor firing times and have no oracle arm.

4.4 End-to-end utility depends on the monitor–recovery pair

The natural-policy recovery deployment covers GPT-5.6 Luna on 40 Evolving Intent tasks. Equal-weighting the six displayed monitoring methods, deterministic re-grounding reaches 75.0% success, the quote-only WATCH note 74.6%, monitored no state action 72.9%, and lossy compaction 59.6%. These averages hide the central interaction. Active recomputation improves from 72.5% with compaction to 85.0% with re-grounding, whereas selected passive methods show the opposite descriptive pattern: compaction exceeds re-grounding by 5.0 points for Trace rules and 2.5 points for Frozen recompute, and Trace judge ties. Thus, neither intervention nor no intervention is uniformly best; the outcome depends on which monitor and recovery operator are paired.

Each policy can act at most once, but natural firing rates are not forced to match, so these recovery rankings remain descriptive. They are also sensitive to task inclusion: a cumulative $n = 38$ omission audit changes 23 scientific-outcome, 12 action-policy, and 18 resource conclusion flags. We therefore report the complete $n = 40$ analysis while treating individual deployment rankings as hypotheses for a powered replication.

5 Discussion / Conclusion

Our results suggest that monitoring long-horizon agents is not solely a prediction problem. Active probes can improve a signal while changing the task trajectory, oracle timing can create genuine headroom while still interacting with recovery, and the same monitoring method can move in opposite directions under lossy compaction and deterministic re-grounding. The appropriate unit of

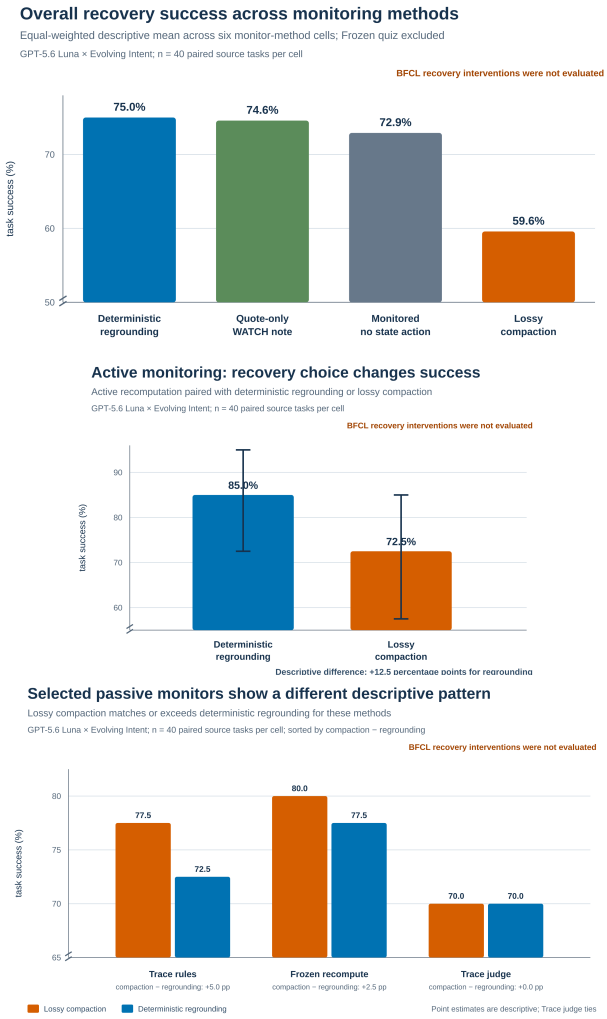

Figure 6: Recovery results in the GPT-5.6 Luna × Evolving Intent deployment ($n = 40$ paired source tasks per cell). *Top:* equal-weighted outcome across six monitoring methods. *Middle:* active recomputation under lossy compaction and deterministic re-grounding, with task-bootstrap 95% intervals. *Bottom:* selected passive monitors under the same two operators. Frozen quiz is excluded, firing rates are natural rather than matched, and BFCL recovery interventions were not evaluated.

evaluation is therefore the complete monitor, intervention policy, and recovery mechanism, measured by downstream task success rather than signal quality alone.

The present recovery deployment is limited to one model, one reasoning benchmark, 40 tasks, unequal natural firing rates, and one intervention per task. Future work should replicate recovery on BFCL and additional models, compare monitor–recovery pairs at matched intervention budgets, and evaluate planning policies that choose whether to intervene as a function of expected recovery loss. More broadly, deterministic state stores introduce their own assumptions; testing stale, incomplete, or corrupted external state would clarify when re-grounding remains preferable to compaction.

References

[1] Atul Anand and Sourav Chattaraj. Diagnosing tool-selection reasoning in LLM agents with canary tools. *arXiv preprint arXiv:2608.04719*, 2026. URL https://arxiv.org/abs/2608.04719.

[2] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

[3] Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. LLMs get lost in multi-turn conversation. *arXiv preprint arXiv:2505.06120*, 2025. URL https://arxiv.org/abs/2505.06120.

[4] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a_00638. URL https://aclanthology.org/2024.tacl-1.9/.

[5] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR, 2025. URL https://proceedings.mlr.press/v267/patil25a.html.

[6] Kai Ruan, Zihe Huang, Ziqi Zhou, Qianshan Wei, Xuan Wang, and Hao Sun. Doomed from the start: Early abort of LLM agent episodes via a recall-controlled probe cascade. *arXiv preprint arXiv:2607.06503*, 2026. URL https://arxiv.org/abs/2607.06503.

[7] Andrew Semenov and Svyatoslav Dorofeev. Beyond compaction: Structured context eviction for long-horizon agents. *arXiv preprint arXiv:2606.11213*, 2026. URL https://arxiv.org/abs/2606.11213.

[8] Yuan Sui, Yulin Chen, Yibo Li, Xue Jiang, Yufei He, Yihong Dong, Xiaoxin He, Tianyu Gao, and Bryan Hooi. TACT: Mitigating overthinking and overacting in coding agents via activation steering. *arXiv preprint arXiv:2605.05980*, 2026. URL https://arxiv.org/abs/2605.05980.

[9] Rakshith Vasudev, Melisa Russak, Dan Bikel, and Waseem Alshikh. Accurate failure prediction in agents does not imply effective failure prevention. *arXiv preprint arXiv:2602.03338*, 2026. URL https://arxiv.org/abs/2602.03338.

[10] Chenyu Wang, Yunbo Lyu, Junda He, Zhou Yang, Chenxing Zhong, Yaniv Harel, and David Lo. Fail-fast, restart-smart: Early failure prediction and restart for SWE agentic tasks. *arXiv preprint arXiv:2608.03222*, 2026. URL https://arxiv.org/abs/2608.03222.

[11] Jason Weston, Antoine Bordes, Sumit Chopra, Alexander M. Rush, Bart van Merriënboer, Armand Joulin, and Tomas Mikolov. Towards AI-complete question answering: A set of prerequisite toy tasks. *arXiv preprint arXiv:1502.05698*, 2015. URL https://arxiv.org/abs/1502.05698.

[12] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=roNSXZpUDN.

[13] Yuxiang Zheng, Dayuan Fu, Xiangkun Hu, Xiaojie Cai, Lyumanshan Ye, Pengrui Lu, and Pengfei Liu. DeepResearcher: Scaling deep research via reinforcement learning in real-world environments. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 414–431. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.22. URL https://aclanthology.org/2025.emnlp-main.22/.

[14] Yu Zhuang, Kefei Chen, Yitong Duan, Shuxin Zheng, Jian Li, and Xu-Yao Zhang. AgentRewind: Recoverable execution for long-horizon LLM agents. *arXiv preprint arXiv:2608.14380*, 2026. URL https://arxiv.org/abs/2608.14380.